

---

## 1. ABOUT THE ORGANIZATION

Accolade Tech Solutions Private Limited is a well-established technology company based in Mangalore, specializing in Mobile App Development, Web Development, Customized Software, Digital Marketing, and Smart TV Applications. With a strong commitment to innovation and excellence, we leverage cutting-edge technologies to develop high-performance solutions tailored to diverse business needs. Our expertise spans secure and scalable Mobile Applications using Flutter, Artificial Intelligence solutions developed in Python, and dynamic, user-friendly Web Applications built with React JS, Node JS, Express JS, PHP, HTML5, and CSS. Our skilled team of Android and iOS developers ensures the creation of powerful mobile applications with seamless functionality, while our cloud computing specialists efficiently manage platforms like AWS, GCP, and other cloud services to provide reliable, scalable, and secure infrastructure for businesses. We also specialize in developing robust e-commerce solutions for both single-vendor and multi-vendor platforms, integrating seamless payment gateways, intuitive user interfaces, and efficient backend systems to enhance the digital shopping experience.

In addition to our expertise in web and mobile development, we are industry leaders in Smart TV application development, crafting high-quality apps for Android TV and Amazon Fire TV. With over 12 years of experience, we have successfully collaborated with TV channels to develop feature-rich news applications for websites, mobile devices, and Smart TVs. Our portfolio also includes innovative OTT solutions and customized applications for niche markets such as dental associations, ensuring businesses receive tailored digital solutions that drive efficiency and engagement. Our long-term commitment to technological sustainability enables us to stay ahead of industry trends, continuously refining our strategies to deliver superior products and services. At Accolade Tech Solutions, we believe in empowering businesses with future-ready solutions that enhance user experience, streamline operations, and create a lasting digital impact.

---

## 2. INTERNSHIP POSITION

**Role:** Intern

Responsibilities:

As an intern, my primary role was to contribute to the development of frontend web applications using React JS and to apply styling with Material-UI. This involved several key tasks:

- **Front-End Development:** I was responsible for building user interfaces using React JS, a popular JavaScript library for creating interactive UIs. This included creating reusable components, managing component state, and implementing responsive layouts to ensure a seamless user experience across different devices.
- **Styling with Material-UI:** I utilized Material-UI, a React component library that adheres to Google's Material Design principles, to style the applications. This task involved working with pre-designed components such as buttons, cards, and grids, as well as customizing themes and components to align with the project's specific design requirements and branding.
- **Component Integration:** I integrated various UI components into cohesive user experiences. This included ensuring that all elements functioned correctly, interacted smoothly with one another, and adhered to the overall design specifications, resulting in a polished and user-friendly application.

---

## 3. INTERNSHIP EXPERIENCE

### 3.1 SKILLS GAINED DURING INTERNSHIP

**React JS:** React is a JavaScript library for building user interfaces, developed by Facebook. It allows developers to create reusable UI components that manage their own state, making it easier to build complex UIs efficiently. React uses a virtual DOM (Document Object Model) for optimal performance by updating only the necessary parts of the actual DOM when data changes. Material-UI is a popular React UI framework that implements Google's Material Design. It provides a set of pre-designed and customizable components such as buttons, cards, menus, and more, which adhere to the Material Design principles. These components help developers maintain a consistent look and feel across their applications while offering flexibility for customization.

**CSS:** Cascading Style Sheets (CSS) is used to define the visual presentation of web pages written in HTML and XML. In React applications, CSS can be applied in various ways: inline styles, CSS modules (scoped CSS), or using preprocessors like Sass or Less. It allows developers to style React components to achieve the desired visual appearance and layout.

**JavaScript:** JavaScript is the core programming language used in React applications for implementing logic, handling events, data fetching, and more. It integrates seamlessly with React to create interactive and dynamic user experiences.

**Bootstrap:** It is a popular open-source front-end framework for developing responsive and mobile-first websites and web applications. It provides a collection of CSS and JavaScript

---

components, such as grids, navigation bars, buttons, forms, modals, and carousels, which help streamline the development process. Bootstrap follows a 12-column grid system for layout and includes pre-designed templates and themes, making it easy to create visually appealing and consistent designs. It is

widely used for its ease of use, extensive documentation, and ability to quickly create responsive layouts that adapt to different screen sizes and devices.

**Material-UI:** Material-UI is a popular React UI framework that adheres to Google's Material Design principles, providing developers with a set of pre-designed and customizable components. These components, including buttons, cards, menus, and more, ensure a consistent and visually appealing look and feel across applications. Material-UI streamlines the process of creating responsive and visually engaging interfaces by offering built-in styling and theming capabilities, allowing developers to focus on building functional and efficient applications. Its flexibility and ease of use make it a go-to choice for React developers aiming to implement Material Design in their projects seamlessly.

**Public API:** In a React application, a public API is accessed by making HTTP requests to fetch or send data from external services. This is typically done using libraries like Axios or the Fetch API within React components. Developers integrate these APIs to retrieve data such as user information, geographical data, or weather updates, which can then be dynamically displayed or processed in the application. The API interaction is often managed within lifecycle

---

methods or hooks like `useEffect` to handle side effects, ensuring that data fetching occurs efficiently and updates the component state accordingly, thus rendering the data in the user interface seamlessly.

**Node JS:** It is a runtime environment that allows developers to execute JavaScript code on the server side, outside of a web browser. Built on Chrome's V8 JavaScript engine, Node.js is designed for building scalable and high-performance applications, particularly suited for I/O-heavy tasks such as real-time applications, RESTful APIs, and server-side applications. It utilizes an event-driven, non-blocking I/O model, which helps handle multiple requests concurrently without waiting for one operation to complete before starting another. Node.js also has a vast ecosystem of libraries and modules available through npm (Node Package Manager), facilitating rapid development and integration of various functionalities.

## **3.2 APPLICATION OF ACQUIRED SKILLS IN ASSIGNED**

### **3.2.1 FOUNDATIONAL SKILLS**

- Created static web pages using HTML and CSS.
- Enhanced pages with JavaScript for interactivity and DOM manipulation

### **3.2.2 DEVELOPMENT ENVIRONMENT**

- Set up environments with Node.js and NPM.
- Built a basic HTTP server and explored server-side concepts.

---

### **3.2.3 REACT PROFICIENCY**

- Developed dynamic UIs using JSX and React components.
- Integrated React Router for multi-page applications.
- Managed state and side effects using hooks like `useState` and `useEffect`.

### **3.2.4 MODERN JAVASCRIPT**

- Utilized features such as arrow functions, destructuring, and array methods like `map()`.

### **3.2.5 UI/UX DESIGN**

- Integrated Material-UI for styling and improved design.

### **3.2.6 DATA HANDLING**

- Fetched data from public APIs using `Axios` and displayed it in React applications.

### **3.2.7 PRACTICAL PROJECTS**

- Developed a frontend web application.

## **3.3 CHALLENGES**

- Adjusting to new technologies and frameworks like React, which may involve understanding component-based architecture, state management with hooks, and JSX syntax, especially for those transitioning from traditional HTML/CSS/JavaScript development.
- Dealing with challenges such as CORS (Cross-Origin Resource Sharing) issues, authentication problems, or

---

handling asynchronous data fetching and state updates when integrating APIs into web applications.

- Ensuring consistent behavior and appearance across different browsers, addressing CSS rendering differences and JavaScript compatibility issues, which can affect the user experience and functionality of web applications.
- Optimizing the performance of web applications, including reducing load times, improving rendering efficiency, and managing memory usage to enhance user experience and meet performance benchmarks.
- Implementing responsive and visually appealing designs, ensuring accessibility, and maintaining consistency across different devices and screen sizes, often involving CSS frameworks or UI libraries.
- Balancing multiple tasks, learning new technologies, and meeting project deadlines, while ensuring productivity and maintaining code quality throughout the internship period.

### **3.4 LITERATURE REVIEW**

To understand the current trends and challenges in frontend development for Cafe Management Systems using ReactJS and Material UI, the following approach was taken:

- Conducted a comprehensive literature review by exploring existing cafe management and restaurant POS websites, including insights from frontend developers involved in such projects.
- Reviewed case studies of cafe and restaurant management applications built with ReactJS and MUI, and gathered user feedback to identify common UI/UX issues and functional requirements.

- 
- Identified frequent challenges faced with modern frontend technologies, such as performance issues when handling dynamic menu displays, order tracking, and real-time inventory updates. Some platforms also encounter delays in processing multiple simultaneous orders and managing data synchronization between the frontend and backend systems.



---

## **4. SOFTWARE REQUIREMENT SPECIFICATION**

### **[SRS]**

#### **4.1 INTRODUCTION**

The Software Requirements Analysis and Specification is a document, which describes completely the external behaviour of the software. The first and foremost work of a software developer is to study the system to be developed and specify the user requirements before going for the designing phase. This document will let us know how this system behaves and responds.

##### **4.1.1 PURPOSE**

The main purpose of this Software Requirements Analysis and Specification is to translate the ideas in the minds of a client into a formal document. Through SRS the client clearly describes what it expects from the proposed system and the developer clearly understands what capabilities are required to build the system. The purpose of this document is to serve as a guide to the developers and testers who are responsible for the development of the system. The main objective of the project is to eliminate duplicate projects which are executed in different colleges, maintain project documentation, explore new ideas and give an opportunity for juniors to know about the project and to manage all the students' marks based on meeting deadline, performance and presentation.

##### **4.1.2 INTENDED AUDIENCE AND READING SUGGESTIONS**

This document is intended for the administration, students, and faculty involved with the Vehicle Assistance Network web application. It aims to clearly explain the core concepts and functionalities of the Vehicle Assistance Network system. The document provides a comprehensive overview to help both developers and

clients understand the project with ease. It also outlines the future objectives and potential enhancements for continued development and usability of the system.

#### **4.1.3 SCOPE**

This Software Requirements Specification (SRS) outlines the requirements for the Vehicle Assistance Network system. It is intended for use by developers and will serve as the basis for validating the final delivered application. Any modifications to the requirements in the future must follow a formal change approval process.

The scope of this project includes the following features:

- It is a web-based application designed for vehicle service centers to streamline service management.
- Customers can easily request vehicle services online, view service status, and receive notifications regarding service approval or rejection.
- Job cards, service details, and billing processes are managed digitally to reduce paperwork and improve efficiency.
- Admins can manage users, assign service appointments, and generate reports for analysis.
- Employees can view assigned services, update service details, and review customer feedback.

#### **4.1.4 DEFINITION, ACRONYMS, AND ABBREVIATIONS**

LAN : Local Area Network

DFD : Data Flow Diagram

SRS : Software Requirement Analysis and Specifications.

SQL : Structured Query Language

RAM : Random Access Memory

ASP : Active Server Page

HTTP : Hyper Text Transfer Protocol

WWW : World Wide Web

#### **4.1.5 OVERVIEW**

In traditional vehicle service processes, customers are required to visit the service center in person to request service, which often involves long waiting times and manual paperwork. This can be inconvenient and time-consuming, especially when customers have limited availability or urgent service needs. Additionally, tracking service details and approvals manually may lead to delays or miscommunication. These challenges can be effectively addressed using an online vehicle assistance system, which provides features such as digital job cards, service request submissions, approval tracking, and timely notifications. This ensures a more efficient and reliable experience for both customers and service providers.

#### **4.2 OVERALL DESCRIPTION**

The following subsections provide a complete overview of the SRS documentation for the product **“Vehicle Assistance Network”**. The document begins with a definition of the overall system and its objectives. It then outlines the external interface requirements, followed by a detailed description of the system's main components and key features. Finally, the document concludes with the non-functional requirements that ensure the reliability, usability, and efficiency of the system.

#### **4.3 FUNCTIONAL REQUIREMENTS**

Functional requirements specify the outputs expected for given inputs. The system enforces correct data entry by validating inputs based on data types and disallowing blanks in mandatory fields. Invalid inputs prompt the user to re-enter correct data, with appropriate error messages displayed for guidance.

This project consists of the following modules:

**Admin Modules**

- **Admin Dashboard:** Provides a visual summary of service requests, active job cards, pending feedback, and vehicle status.
- **Manage Users:** Allows the admin to add, update, or delete user accounts (employees and customers), and view their details.
- **Manage Vehicles:** Facilitates registering new vehicles, updating their information, and associating them with customer records.
- **Manage Job Cards:** Enables the creation and management of job cards with vehicle and service details.
- **Manage Services:** Allows admins to view and update service records, assign employees, and monitor progress.
- **Manage Feedback:** Displays user feedback and provides options to respond or take corrective actions.
- **Reports:** Generates comprehensive reports on vehicle services, job card activity, and user engagement.
- **Settings:** Allows admin to change account settings including password and contact details.

**User (Client) Modules**

- **Request Service:** Enables customers to submit service requests by entering vehicle number, problem description, and preferred date.
- **View Status:** Allows users to check the approval status of their submitted service requests and view assigned job card details.
- **Submit Feedback:** Lets customers provide feedback regarding service quality, timeliness, and satisfaction.
- **Account Settings:** Enables users to update their account details and change their password securely.

## Employee Modules

- **View Service Requests:** Displays pending and approved service requests assigned to the employee.
- **Update Job Card:** Allows employees to fill in service completion details, update hours worked, and confirm vehicle delivery.
- **View Feedback:** Enables employees to review client feedback on completed services.
- **Settings:** Lets employees update their credentials and contact details securely.

## 4.4 NON FUNCTIONAL REQUIREMENTS

### 4.4.1 PERFORMANCE REQUIREMENTS

The Vehicle Assistance Network must operate efficiently, ensuring smooth and error-free functionality with built-in validation and feedback mechanisms.

Minimum system requirements include:

- Memory: At least 50 GB of disk space.
- RAM: Minimum 2 GB (4 GB recommended for smoother performance).
- Internet: Stable internet connection is required for all users.
- Browser: Compatible with modern web browsers.
- Server: XAMPP-based backend with MySQL integration.

### 4.4.2 DESIGN CONSTRAINTS

- All form fields must enforce valid data formats (e.g., email, phone number).
- Mandatory fields must be filled before submission.
- A relational database (MySQL) is required.

- 
- The system shall be developed using React.js, Node.js (Express), and MySQL.
  - Appropriate error messages must be displayed for incorrect inputs.

#### **4.4.3 SOFTWARE QUALITY ATTRIBUTES**

- Availability: The application shall be accessible online 24x7 with support for multiple users concurrently.
- Security: Role-based access control must be implemented; admin manages user roles and sensitive operations.
- Maintainability: System design and documentation ensure easy future updates or maintenance.
- Portability: As a web application, it is accessible from various platforms and devices.
- Timeliness: System responses and operations should complete within minimal time to maintain user efficiency and satisfaction.

### **4.5 EXISTING SYSTEM**

In the existing system, vehicle service management is largely manual and paper-based. Customers need to visit the service center in person to request services, fill out physical forms, and wait in queues to get their vehicles serviced. Job card entries, service approvals, and billing are all handled through physical registers or spreadsheets, which are inefficient and prone to errors.

One major drawback of this approach is the risk of data loss due to damaged or misplaced paper records. Tracking service history, follow-ups, and vehicle-related data becomes cumbersome and unreliable. Additionally, during busy service periods, manual coordination often leads to confusion in scheduling, service delays, and lack of communication between the service center and the customers.

This method not only consumes a lot of time and resources but also reduces customer satisfaction and makes the service process less transparent and harder to manage.

## **4.6 PROPOSED SYSTEM**

The proposed Vehicle Assistance Network is a web-based application designed to eliminate the drawbacks of the manual service process and improve overall efficiency. The system provides a digital platform for customers, employees, and administrators to manage the entire vehicle service workflow online.

Customers can register through the system, add their vehicle details, and request service appointments at their convenience. Upon submitting a request, the service center can either approve or reject it based on workload and vehicle condition. Once approved, a digital job card is generated and assigned. The customer receives timely updates regarding the service status and expected delivery date.

The admin can oversee the full operation — from managing user and vehicle records to tracking service progress and generating reports. Employees can log in to view assigned tasks, update service details, and close job cards upon service completion.

This system reduces paperwork, minimizes human errors, saves time, and provides a transparent and reliable way to manage vehicle service requests. All information is stored securely in a centralized database for future reference, ensuring continuity and traceability.

### **4.6.1 MOTIVATION**

In many service centers today, there is no structured digital platform to manage customer requests and service records. Customers often face

difficulties such as long wait times, missed updates, and a lack of clarity regarding service status. Employees manually track job cards and vehicle issues, which can lead to inefficiencies, especially during peak hours.

The need to streamline the service process, reduce physical paperwork, and improve communication between customers and service providers has motivated us to propose the Vehicle Assistance Network. This system is intended to enhance user experience, improve service delivery, and bring automation and transparency into vehicle service management.



---

## **5. SYSTEM DESIGN**

### **5.1. INTRODUCTION**

System design is the process or art of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. One could see it as the application of systems theory to product development. There is some overlap with the disciplines of systems analysis, systems architecture and systems engineering.

In System design focus is on deciding which modules are needed for the system, the specifications of these modules should be interconnected is called system design. System design is also called top-level design. Here we consider a system to be set of components with clearly defined behavior that interact with each other in a fixed manner to produce some behavior. In a system design, the design consists of module definitions, with each module supporting a functional abstraction.

### **5.2. APPLICABLE DOCUMENTS**

The document used for preparing system design is software requirement specification (SRS).

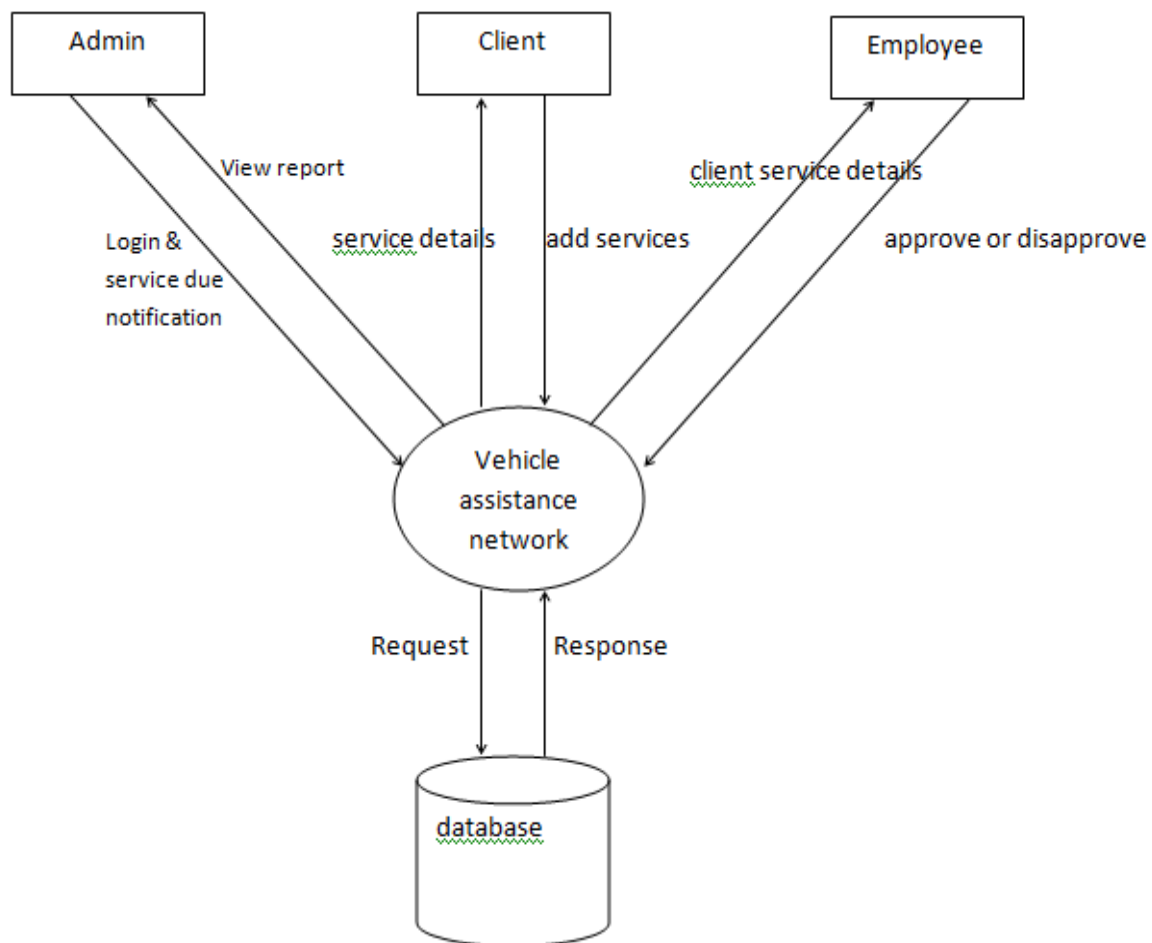
### **5.3. CONTEXT FLOW DIAGRAM**

A context diagram is a data flow diagram with only massive central process that subsumes everything inside the scope of the system. It shows how the system will receive and send data flows to the external entities involved.

Context diagram is a data flow diagram showing data flows between a generalized application within the domain and other entities and abstraction with which it communicates. One thing that differentiate the use of data flow diagram in domain analysis from other typical users is that the variability of the data flow

across the domain boundary must be accounted for which either a set of diagram or text describing the differences.

It is common practice to draw the context-level data flow diagram first, which shows the interaction between the system and external agents which acts as data source and data sinks. On the context diagram the system's interactions with the outside world are modeled purely in terms of data flows across the system boundary. The context diagram shows the entire systems the single process, and gives no clues as to its internal organization.

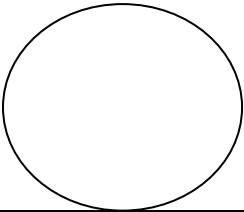
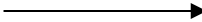
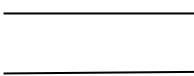


**Fig 5.1 Context Flow Diagram**

In our project, the Administrator of the Vehicle Assistance Network is responsible for registering the personal details of customers and employees. The administrator stores essential information such as contact numbers, vehicle details, and user credentials for future communication and tracking. The system maintains a centralized database that includes all customer records, employee data, and service-related information. The admin performs operations such as adding, updating, or deleting users, vehicles, and job card entries. When a customer submits a service request, the system interacts with the database to process the request, assign it to an employee, and update the job card and service status accordingly. Notifications regarding service approval, progress, or completion can also be sent to the customer's registered contact for real-time updates.

## 5.4. DATA FLOW DIAGRAM

### 5.4.1 NOTATION OF THE DFD

| SYMBOLS                                                                             | DESCRIPTION                                                            |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------|
|  | Represents Source or destination of data                               |
|  | Represents a process that transforms Incoming data into Outgoing flows |
|  | Represents data flow                                                   |
|  | Represents data stores                                                 |

### 5.4.2 LEVEL 1: DATA FLOW DIAGRAM

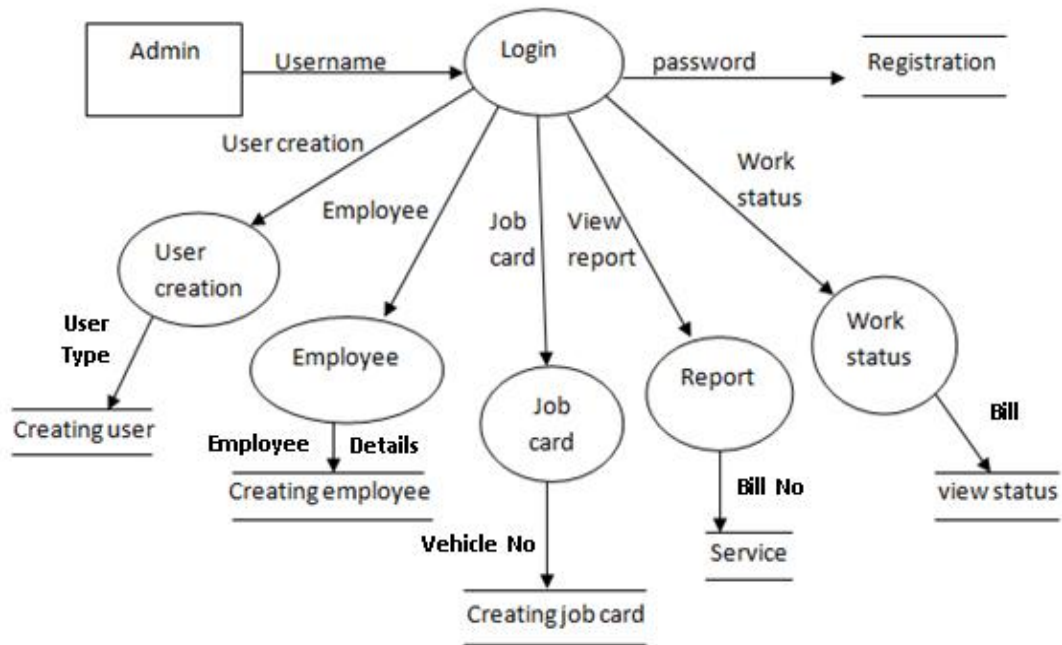


Fig.5.2 Level 1 DFD

### 5.4.3 LEVEL 2: DATA FLOW DIAGRAM

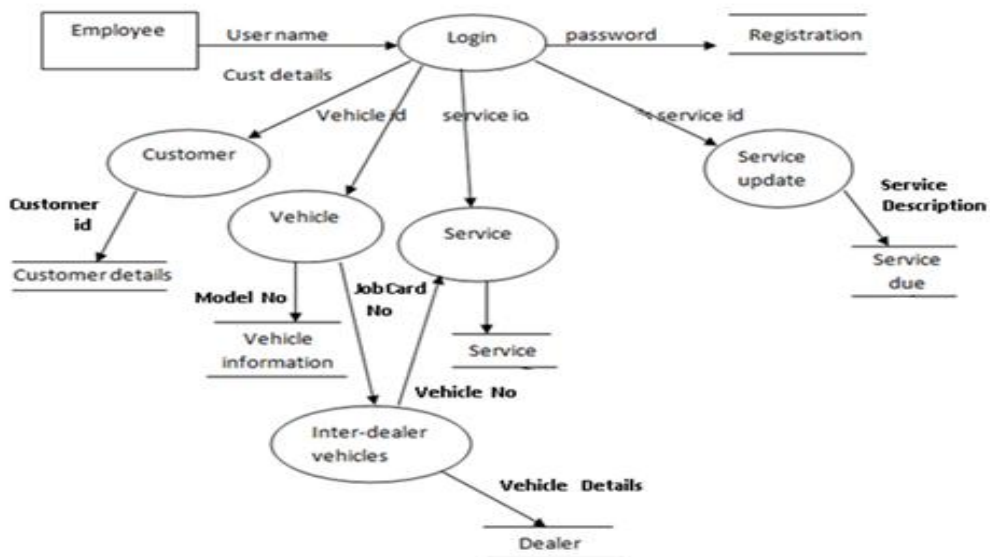
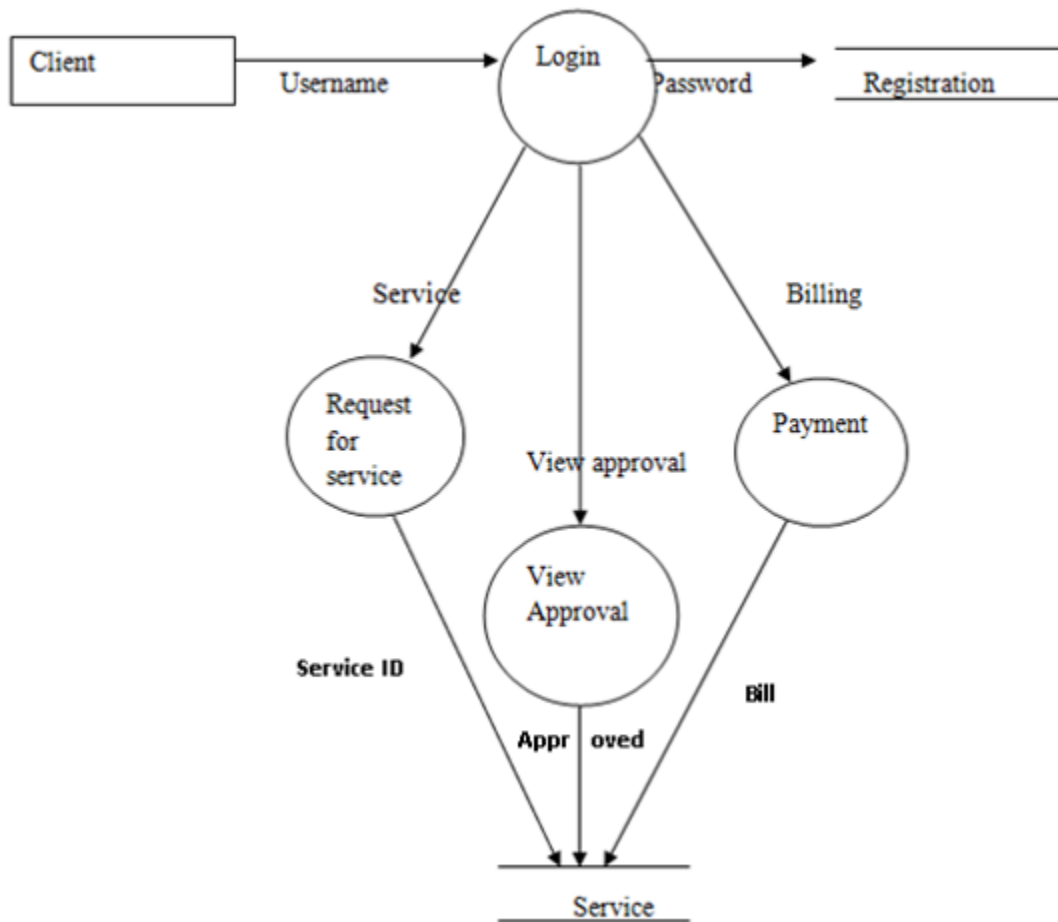
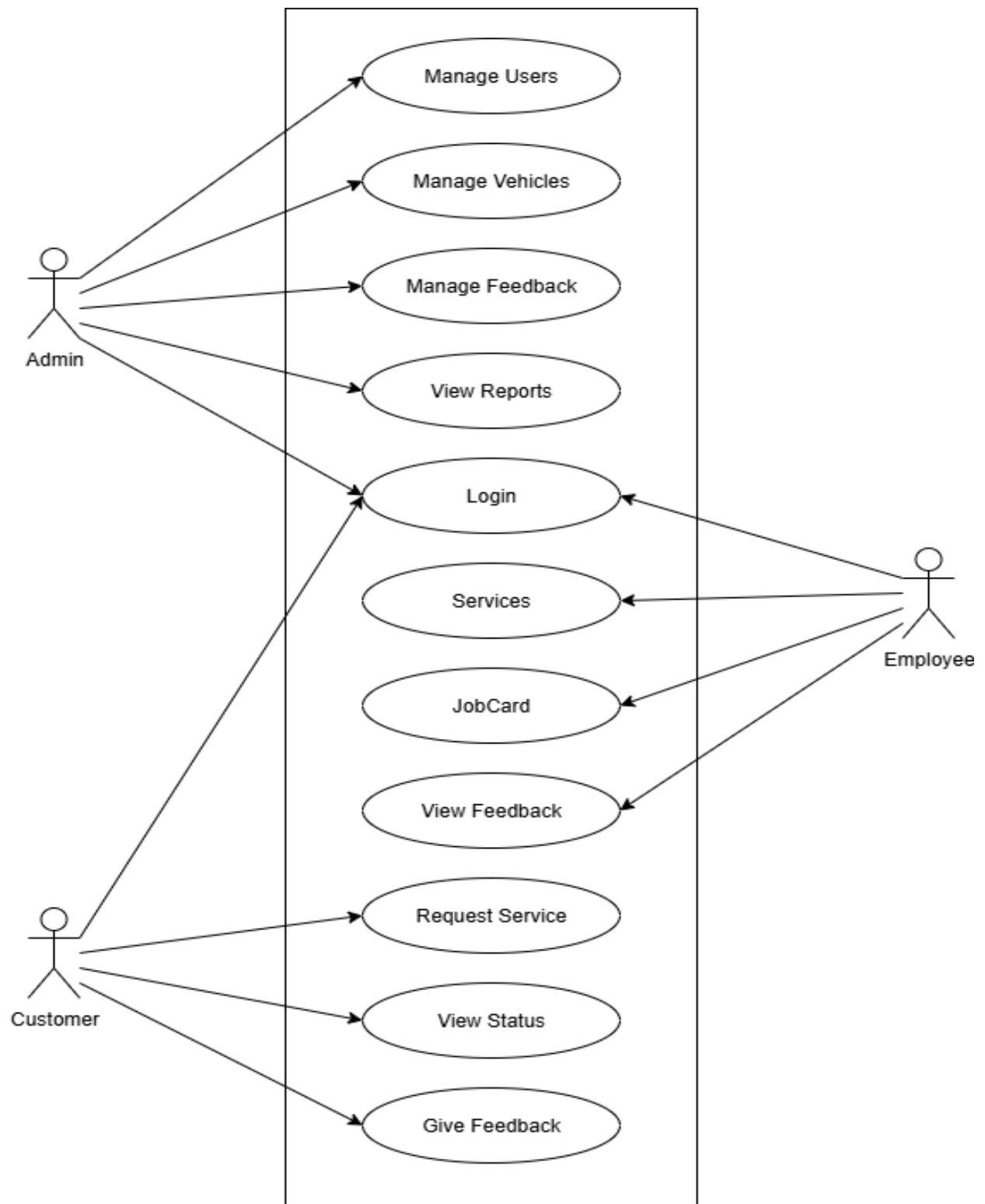


Fig. 5.3 Level 2 DFD

**5.4.4 LEVEL 3: DATA FLOW DIAGRAM****Fig. 5.4 Level 3 DFD**

## 5.5 USE CASE DIAGRAM



**Fig. 5.5 Use Case Diagram**

---

## **6. DATABASE DESIGN**

### **6.1. INTRODUCTION**

Database description describes all the databases used in the software to store all the records. The database in turn is further described in detail giving all the fields used with their data type, constraints available to them and description. Constraints include primary key, foreign key, etc., which allow the entities to be uniquely identified.

In this database description we describe all databases which are used to store all the records of the Canteen System.

#### **6.1.1. INTERNAL LEVEL**

The internal level has an internal schema. It describes the physical storage structure of the database. The internal schema uses a particular data model and describes the complete details of the storage and access paths of the database.

#### **6.1.2. CONCEPTUAL LEVEL**

The conceptual level has conceptual schema, which describes the structure of whole database for a community of users. The conceptual schema hides the details of the physical storage structure and concentrates on describing entities, datatype relationship user operation and constraints. A high-level data model or an implementation data model can be used in this model.

#### **6.1.3. EXTERNAL LEVEL**

The external level or view includes a number of external schema or view. Each internal schema describes the part interested in and hides the rest of the database from the user group. A high-level data model or an implementation model can be used at this level.

## 6.2. DATABASE

A Database is collection of related data, which can be of any size and complexity. By using the concept of Database, we can easily store and retrieve the data. The major purpose of a database is to provide the information, which utilizes it with the information's that the system needs according to its own requirements.

## 6.3. DATABASE DESIGN

Database design is done before building it to meet needs of end-users within a given information-system that the database is intended to support. The database design defines the needed data and data structures that such a database comprises. The database is physically implemented using MySQL.

### 6.3.1 SCHEMA DIAGRAM

**Registration:**

**Vehicle-Details**

**Table:6.1 Vehicle-Details**

| Attribute        | Datatype    | Constraints |
|------------------|-------------|-------------|
| Vehicle_ID       | INTEGER     | Primary key |
| Customer_ID      | VARCHAR(10) | Not Null    |
| Vehicle_Number   | VARCHAR(10) | Not Null    |
| Model_Number     | VARCHAR(20) | Not Null    |
| Insurance_Number | VARCHAR(20) | Not Null    |



---

## Job Card

**Table: 6.2 Job Card**

| Attribute    | Datatype    | Constraints |
|--------------|-------------|-------------|
| JobCardID    | INTEGER     | Primary key |
| VehicleNo    | VARCHAR(10) | Not Null    |
| CustomerName | VARCHAR(25) | Not Null    |
| EmployeeName | VARCHAR(25) | Not Null    |
| Problem      | VARCHAR(45) | Not Null    |

## Service

**Table:6.3 Service**

| Attribute     | Datatype    | Constraints |
|---------------|-------------|-------------|
| Service_ID    | INTEGER     | Primary key |
| Bill_No       | VARCHAR(10) | Not Null    |
| Vehicle_No    | VARCHAR(10) | Not Null    |
| Customer_Name | VARCHAR(25) | Not Null    |
| Employee_Name | VARCHAR(25) | Not Null    |
| Jobcard_No    | VARCHAR(10) | Not Null    |
| Amount        | VARCHAR(10) | Not Null    |
| Description   | VARCHAR(25) | Not Null    |

---

|            |             |          |
|------------|-------------|----------|
| Problem    | VARCHAR(45) | Not Null |
| No of Days | VARCHAR(10) | Not Null |
| Approval   | VARCHAR(25) | Not Null |

## Request for service

**Table:6.4 Request for service**

| Attribute         | Datatype    | Constraints |
|-------------------|-------------|-------------|
| Service_approvaID | INTEGER     | Primary key |
| Jobcard_No        | VARCHAR(10) | Not Null    |
| Vehicle_No        | VARCHAR(10) | Not Null    |
| Customer_Name     | VARCHAR(25) | Not Null    |
| Date              | VARCHAR(10) | Not Null    |
| Approved          | VARCHAR(25) | Not Null    |

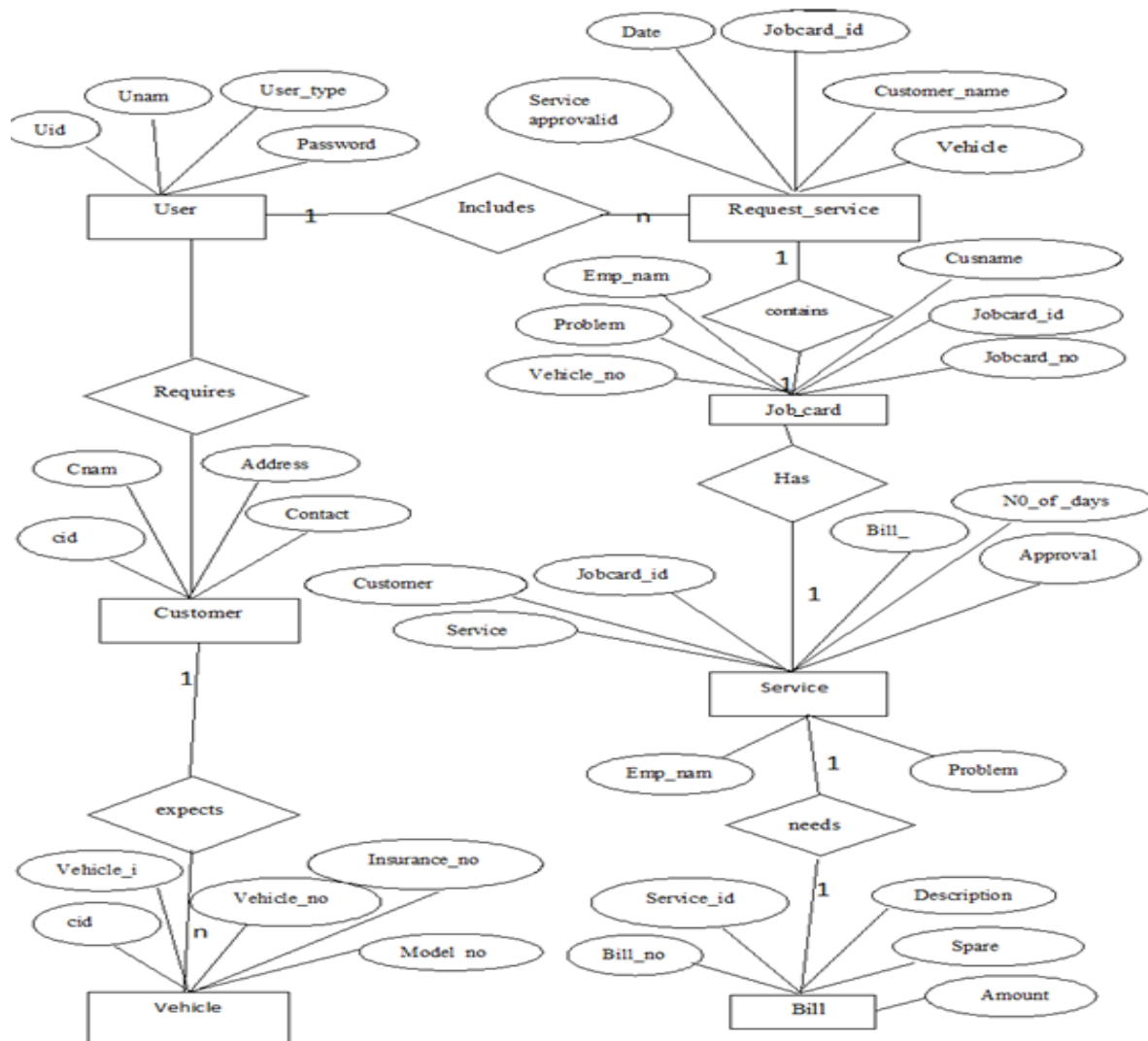
## Billing

**Table:6.5 Billing**

| Attribute   | Datatype    | Constraints |
|-------------|-------------|-------------|
| Bill_No     | INTEGER     | Primary key |
| Service_id  | VARCHAR(10) | Not Null    |
| Date        | VARCHAR(10) | Not Null    |
| Amount      | VARCHAR(10) | Not Null    |
| Description | VARCHAR(35) | Not Null    |

## 6.4. ENTITY- RELATIONSHIP (ER) DIAGRAM

An entity-relationship diagram is a specialized graphic that illustrates the relationships between entities in a database. ER diagrams often use symbols to represent three different types of information. Boxes are commonly used to represent entities. Diamonds are normally used to represent relationships and ovals are used to represent attributes.



**Fig. 6.6 ER Diagram**

## **7. DETAILED DESIGN**

### **7.1 INTRODUCTION**

Detailed design starts after the system design phase and system has been certified through the review. The goal of this phase is to develop the internal logic of each of the modules identified during system design.

In the system design, the focus is on identifying the modules, whereas during detailed design the focus is on designing the logic for the modules. In other words, in system design attention is on what components are needed, while in the detailed design how the components can be implemented in the software is the issue.

The design process for software system has two eves. At the first level focus is on deciding which modules are needed for the system, the specification of these modules and how the modules should be interconnected. This is called system design or top-level design. In the specification of the module can be satisfied is decided. This design level is often called detailed design or logic design.

Because the detailed design is extension of system design, system design controls the major structural characteristics of the system. The system design has a major impact testability and modifiability of a system and impacts its efficiency much of the design efforts for the designing software are spent creating the system design.

#### **7.1.1. APPLICABLE DOCUMENTS**

The documents used during detailed design are:

- System Requirements Document
- System Design
- Database Design

### 7.1.2. STRUCTURE OF THE SOFTWARE PACKAGE

The functional components identified during the system design are listed here:

Functional Component 1: Employee

Functional Component 2: Job Card

Functional Component 3: Customer Details

Functional Component 4: Service

Functional Component 5: Vehicle Details

Functional Component 6: Request for Service

Functional Component 7: Billing

## 7.2 MODULAR DECOMPOSITION OF COMPONENTS

### 7.2.1 ADMIN COMPONENT

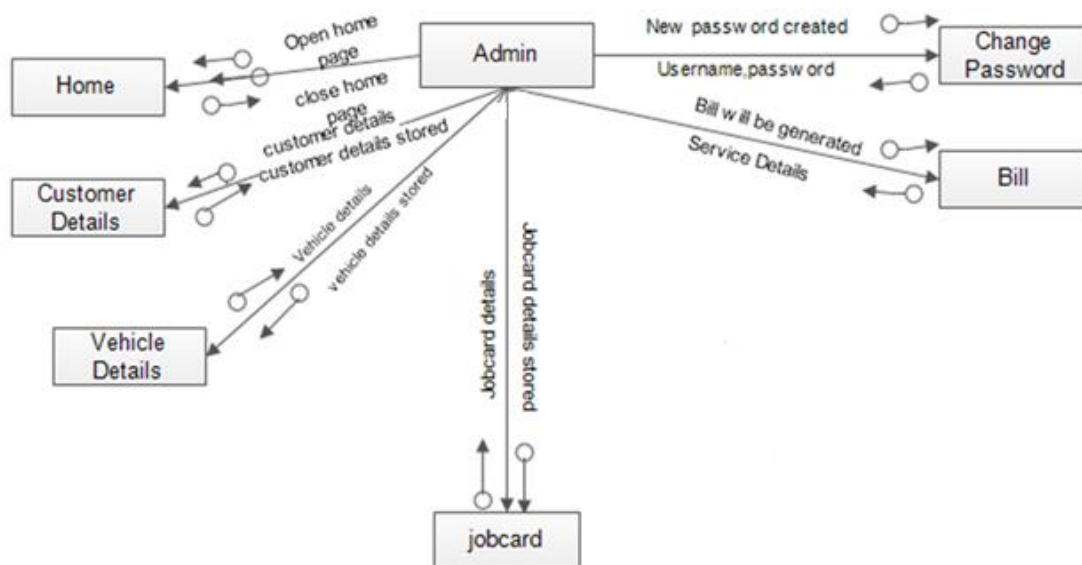


Fig. 7.1 Admin Component

### 7.2.2 EMPLOYEE COMPONENT

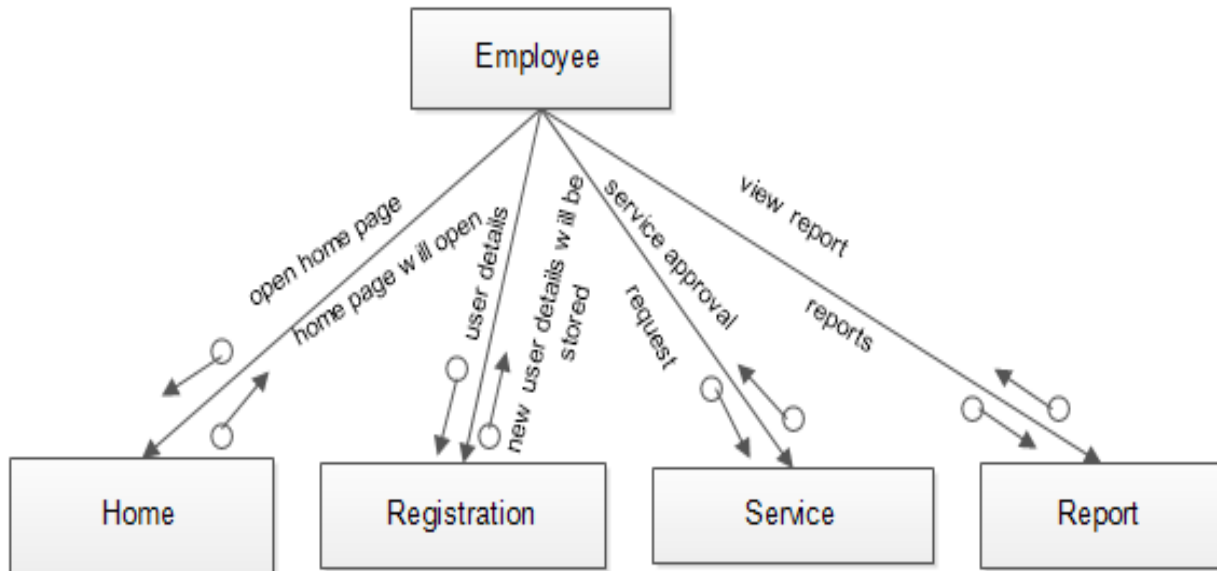


Fig. 7.2 Employee Component

### 7.2.3 CUSTOMER COMPONENT

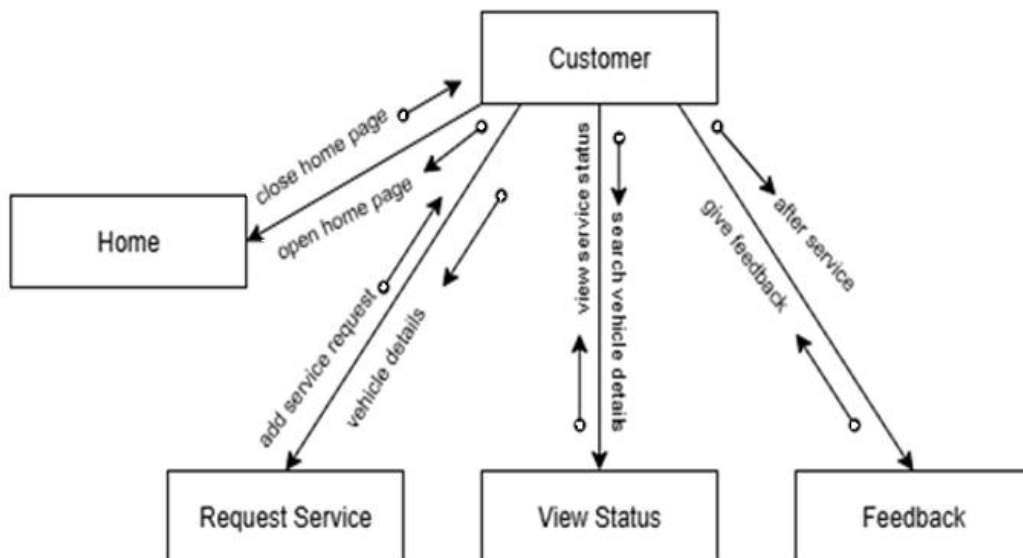


Fig. 7.3 Customer Component

---

## 8. IMPLEMENTATION

As part of my internship, I was involved in building some of the important modules for the Cafe Management System. The frontend was developed using React JS and styled with Material UI, while MySQL was used to store and manage the application's data.

### 8.1 MODULES I WORKED ON:

#### 8.1.1 Admin Side:

- **AddUser:**

The Add User module enables admins to create accounts for clients and employees, assigning unique usernames, passwords, and user types (Client or Employee). I developed a clean form interface using React JS and Material UI components (e.g., Text Field, Button), ensuring a professional look and feel. The module uses JavaScript to handle form submissions and API calls to store user data in the MySQL database, with validation to prevent duplicate usernames or invalid inputs, ensuring secure and efficient user management.

- **AddVehicle:**

In the Add Vehicle module, admins can register vehicle details, including Vehicle ID, Customer Name, Vehicle Number, Model Number, and Insurance Number. I created a form-based interface with Material UI's Grid and Table components, displaying vehicle data in a structured format. JavaScript was used to manage state with React hooks (use State, use Effect) and integrate with MySQL via API calls for data storage and validation, preventing errors like duplicate vehicle numbers.

- **ManageFeedback:**

The Manage Feedback module allows admins to view, respond to, and delete client feedback. I built a responsive table interface using Material UI's DataGrid, displaying Feedback ID, Customer Name, and Comments

---

with action buttons for management. JavaScript handled API requests to fetch and update feedback records in the MySQL database, with real-time UI updates using React hooks for a seamless admin experience.

- **Reports:**

The Reports module enables admins to generate insights on service requests, job cards, and feedback. I developed an interface using React JS and Material UI, featuring tables and charts (e.g., via Chart.js) with filtering options by date or customer. JavaScript managed data fetching from MySQL via APIs, with backend SQL queries aggregating data for performance metrics and customer satisfaction reports, supporting informed decision-making.

### 8.1.2 User (Client) Side:

- **RequestService:**

The Request Service module allows clients to submit vehicle service requests with details like Vehicle Number, Problem Description, and Submission Date. I created a user-friendly form using Material UI's Text Field and Button components, styled for clarity. JavaScript and React hooks handled form validation and API calls to store requests in the MySQL database, sending notifications to the service center for approval or rejection, with a confirmation message displayed upon submission.

- **ViewStatus:**

In the View Status module, clients can check the approval status of their service requests, including Service Approval ID, Job Card Number, and Approval Status (Approved/Rejected). I developed a card-based layout using Material UI's Card component, with data fetched from MySQL via JavaScript API calls. React hooks ensured real-time status updates, providing a responsive and interactive experience for clients.



- **Feedback:**

The Feedback module enables clients to submit feedback on their service experience, including comments and ratings. I built a form interface with Material UI's Text Field and Rating components, ensuring a visually appealing design. JavaScript managed form submissions and API integration with MySQL, with client-side validation to prevent empty or invalid entries, enhancing user engagement with a confirmation message upon submission.

### 8.1.3 Employee Side:

- **ViewService:**

The View Service module allows employees to review service requests, including Service ID, Vehicle Number, and Problem Description. I created a table interface using Material UI's DataGrid, with JavaScript fetching data from MySQL via APIs. The module supports approving or rejecting requests, with React hooks managing real-time UI updates and ensuring a seamless experience for employees.

- **JobCard:**

In the Job Card module, employees can view and manage job card details, including Vehicle Number, Customer ID, Employee Name, and Problem. I developed a responsive table interface using Material UI, with JavaScript handling API calls to MySQL for data retrieval and updates. The module ensures efficient service tracking with validation for consistent job card data.

- **ViewFeedback:**

The View Feedback module enables employees to review client feedback related to their services. I built a table interface with Material UI's DataGrid, displaying Feedback ID and Comments, sortable by date or customer. JavaScript managed data fetching from MySQL, with React

hooks ensuring a responsive and intuitive interface for employees to assess service quality.

- **Settings:**

The Settings module allows employees to update their account details, such as passwords or contact information. I developed a form interface with Material UI's TextField and Button components, with JavaScript handling validation for inputs like Current Password, New Password, and Confirm Password. The module integrates with MySQL via APIs to securely update employee records, with error handling for invalid inputs.

---

## 9. TOOLS AND TECHNOLOGIES USED

While developing the Cafe Management System, I worked with a variety of programming languages, frameworks, and tools that made the development process smoother and more organized. Below are the key technologies and tools I used during the project:

### 1. Programming Languages

- **JavaScript:** JavaScript was the primary language used for writing the frontend logic of the application. It helped me create interactive pages and handle user actions effectively.
- **SQL:** I used SQL for handling database operations in MySQL. It helped in storing, updating, and retrieving data like user details, menu items, orders, and wallet balances.

### 2. Frameworks and Libraries

- **React JS:**

React JS was used to build the frontend of the application. Its component-based approach made it easier to create reusable parts of the UI and manage data within the application using state and hooks.

- **Material UI:**

For designing the user interface, I used Material UI. It provided a wide range of ready-to-use components, which made the frontend look clean and modern without having to write too much custom CSS.

### 3. Development Tools

- **Visual Studio Code (VS Code):**

VS Code was my go-to code editor throughout the project. Its useful extensions and features like auto-suggestions and debugging tools made development more efficient.

- **MySQL Workbench:**

To interact with the database, I used MySQL Workbench. It provided an easy way to manage tables, run SQL queries, and check the data stored.

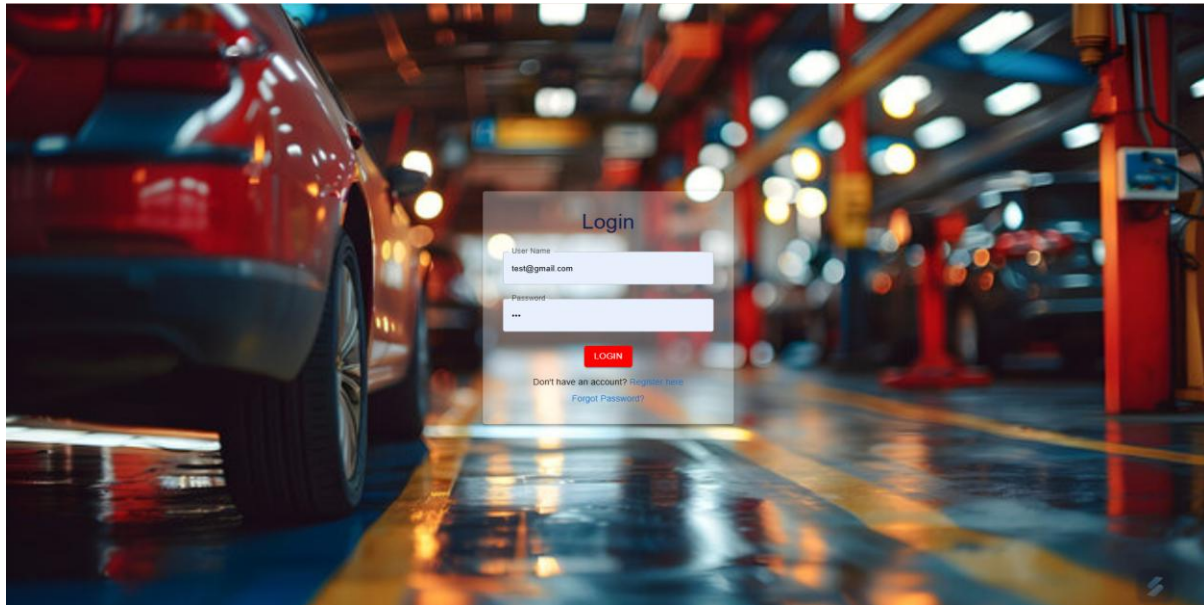
- **Postman:**

Postman was handy for testing APIs (wherever backend interactions were needed). It allowed me to verify that the data being sent and received between the frontend and database was accurate.

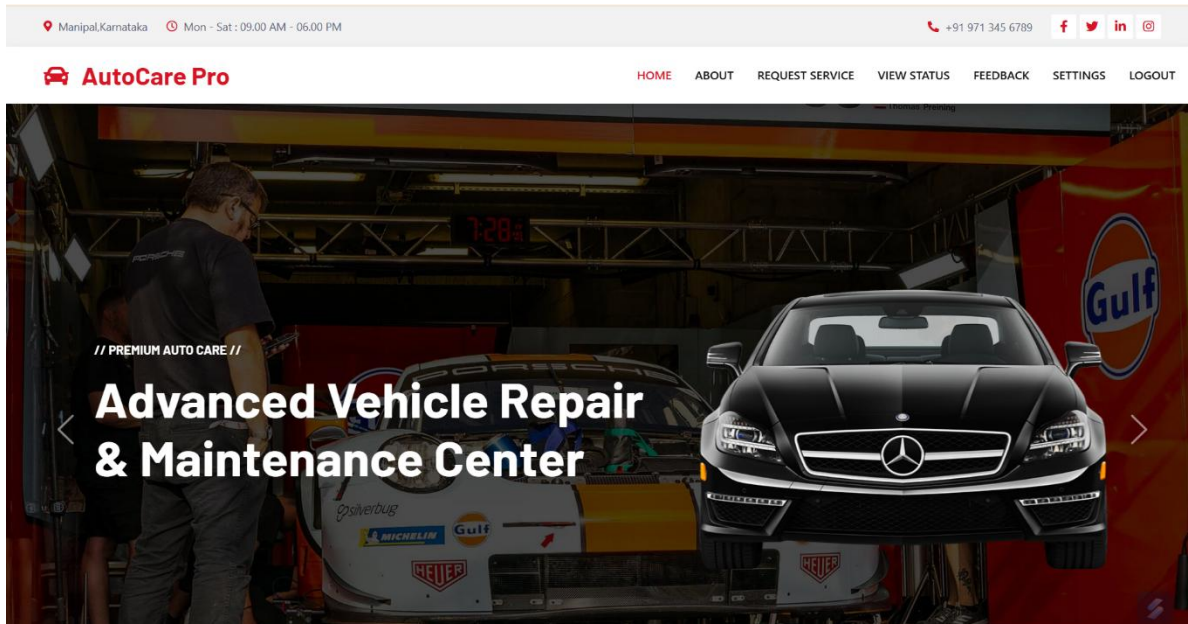
These tools and technologies played a big role in helping me build a functional, organized, and user-friendly Vehicle Assistance Network

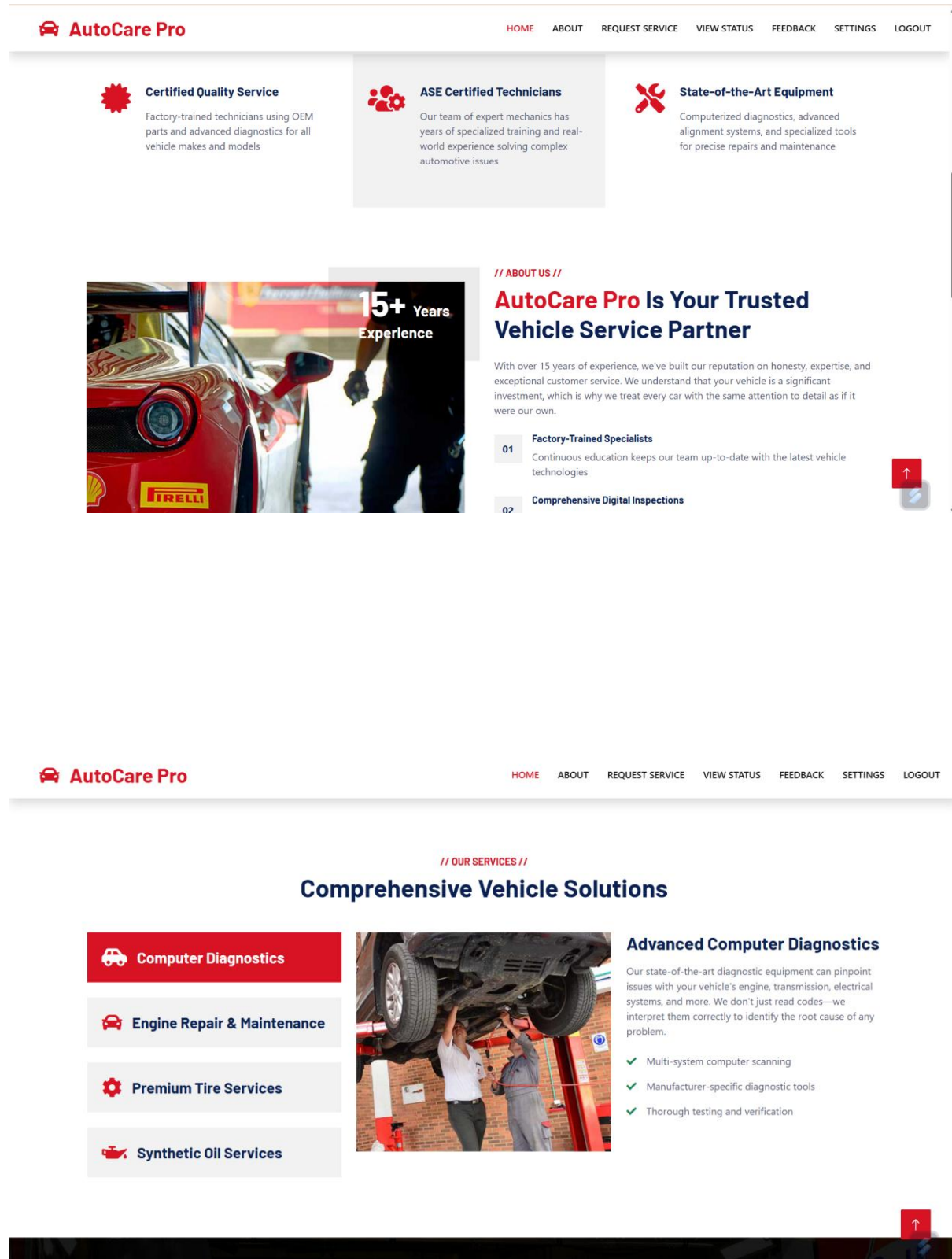
## 10. SCREENSHOTS

### 1. Register/Login Page:



### 2. Home page:





### 3. Request Service:

### Request for Services

SEARCH

REQUEST SERVICE

#### Our Location

Manipal, Karnataka

+91 971 345 6789

service@autocarepro.com

#### Service Hours

**Monday - Friday:**  
9:30 AM - 6:00 PM

**Saturday:**  
10:00 AM - 3:00 PM

**Sunday:**  
Closed

#### Quick Links

- > Home
- > About Us
- > Request Service
- > View Status
- > Feedback

### 4. Feedback:

Manipal,Karnataka Mon - Sat : 09.00 AM - 06.00 PM

+91 971 345 6789

**AutoCare Pro** [HOME](#) [ABOUT](#) [REQUEST SERVICE](#) [VIEW STATUS](#) [FEEDBACK](#) [SETTINGS](#) [LOGOUT](#)

### Feedback

RAHUL

rahulashok5685@gmail.com

Write Feedback

Good Service, Very Impressive Work

SUBMIT FEEDBACK

## 5. Settings:

Manipal,Karnataka

Mon - Sat : 09:00 AM - 06:00 PM

+91 971 345 6789





[HOME](#) [ABOUT](#) [REQUEST SERVICE](#) [VIEW STATUS](#) [FEEDBACK](#) [SETTINGS](#) [LOGOUT](#)

Change Password

CHANGE PASSWORD

## 6. Employee Dashboard:

Dashboard

Services

Job Card

View Feedbacks

Settings

LOGOUT

Employee Dashboard

+12%

7

Total Appointments

+8%

6

Services Completed

+15%

10

Active Customers

Recent Appointments

bhavya - 963963 (KA21N3222)  
test • ID: 46499 Completed4/28/2025, 4:29:41 PM

fayaz - 963963 (KA21Z555)  
invoice test • ID: 85180 Completed4/4/2025, 2:43:29 PM

fayaz - 963963 (KA21Z555)  
regular • ID: 36859 Completed4/4/2025, 10:36:31 AM

ajay - Suzuki (null)  
Regular Maintenance • ID: Scheduled3/18/2025, 11:19:47 AM

rahul - Suzuki (null)  
hyy • ID: 82186 Completed3/17/2025, 5:30:00 AM

Service Performance

Other Services4/4100%

Vehicle Inspection1/1100%



## 7. Service:

Dashboard

Services

Job Card

View Feedbacks

Settings

LOGOUT

### Services

| Sl No | Vehicle Number | Description  | Requested Date | Action               |
|-------|----------------|--------------|----------------|----------------------|
| 1     | KA21N3222      | Break Fixing | 20/07/2025     | <a href="#">VIEW</a> |

## 8. Jobcard:

Dashboard

Services

Job Card

View Feedbacks

Settings

LOGOUT

### Job Card Management

Search by Job Card Number:

[Search](#)

#### Job Card Details

Job Card Number: 34646  
Vehicle Number: KA21Z100  
Job Starting Time: 06:11 PM

#### Customer Details

Name: test  
Address: kulshekar  
Contact: 08521478521  
Email: vignesh@gmail.com  
Vehicle No: KA21Z100

#### Update Job Card

Ending Time:

Number of Hours:

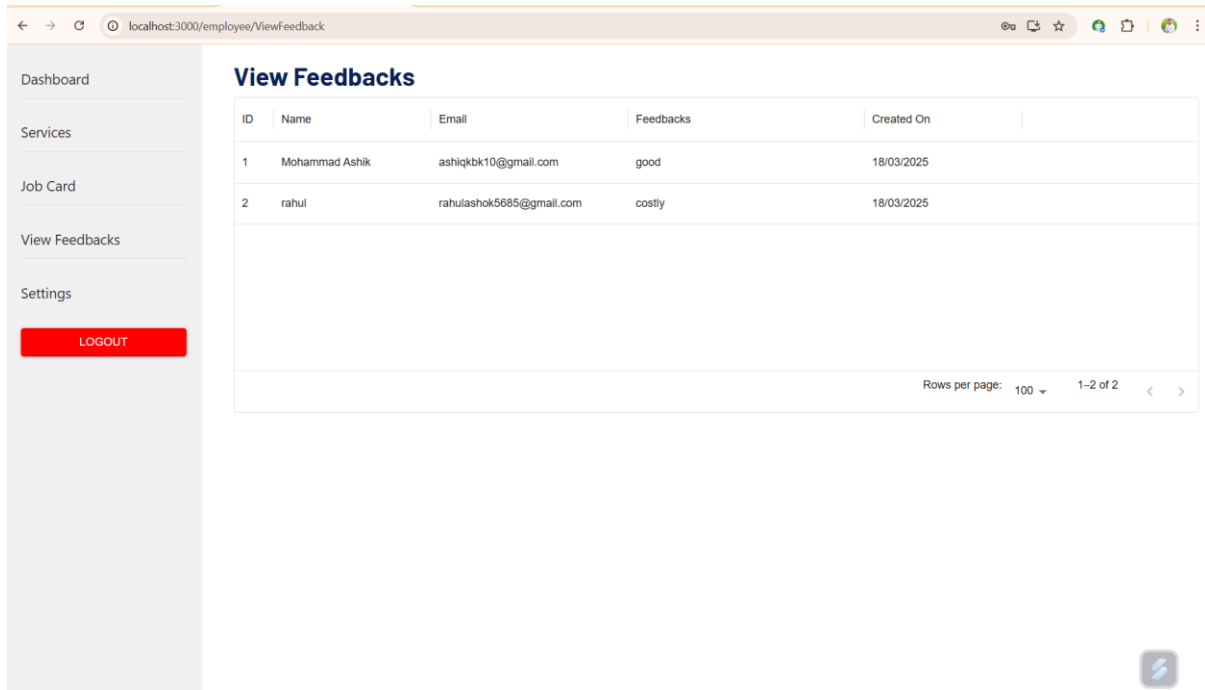
Gross Total (₹):

GST (%):

Net Total (₹):

Problem Description:

## 9. View Feedback:

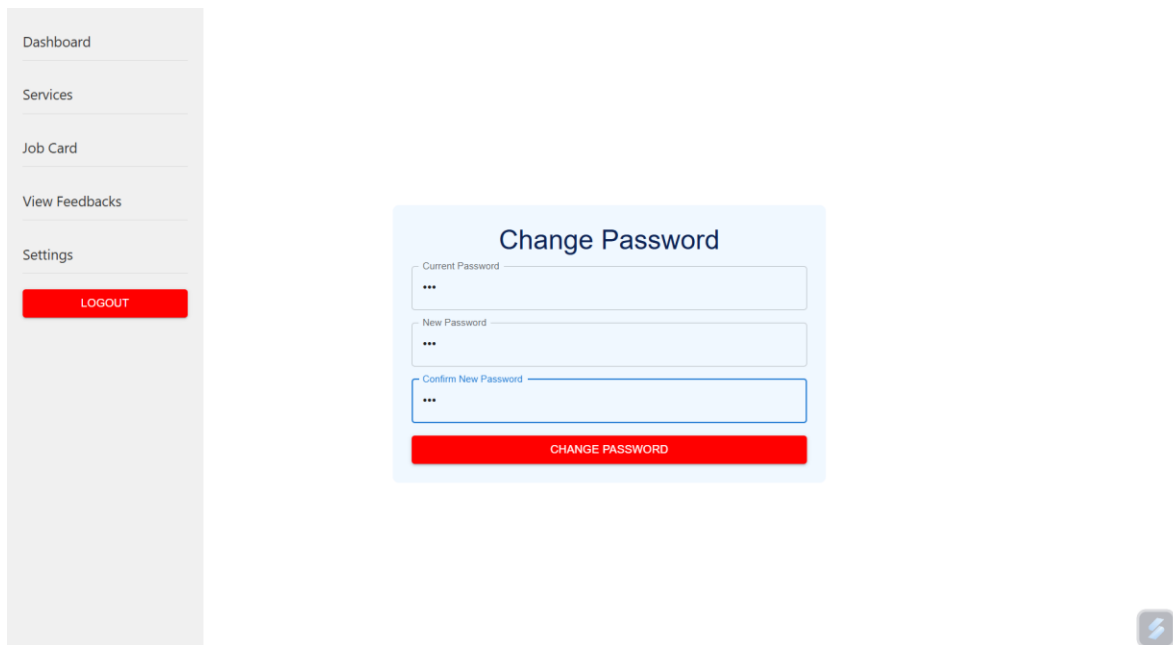


The screenshot shows a web application interface for viewing feedback. The browser address bar indicates the URL is `localhost:3000/employee/ViewFeedback`. The sidebar on the left contains the following links: Dashboard, Services, Job Card, View Feedbacks, and Settings. A red 'LOGOUT' button is located below the Settings link. The main content area is titled 'View Feedbacks' and displays a table with the following data:

| ID | Name           | Email                    | Feedbacks | Created On |
|----|----------------|--------------------------|-----------|------------|
| 1  | Mohammad Ashik | ashiqkbk10@gmail.com     | good      | 18/03/2025 |
| 2  | rahul          | rahulashok5685@gmail.com | costly    | 18/03/2025 |

At the bottom right of the table, there is a pagination control showing 'Rows per page: 100' and '1-2 of 2'.

## 10. Settings:

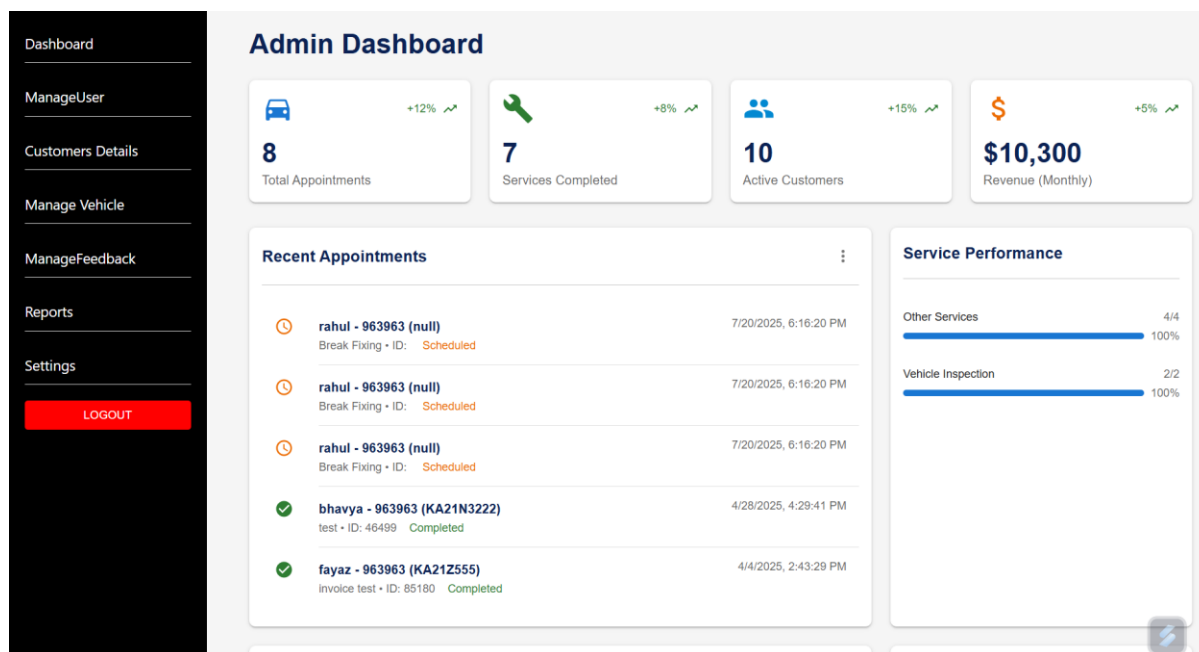


The screenshot shows a web application interface for changing a password. The sidebar on the left contains the following links: Dashboard, Services, Job Card, View Feedbacks, and Settings. A red 'LOGOUT' button is located below the Settings link. The main content area displays a 'Change Password' form with the following fields:

- Current Password (password type)
- New Password (password type)
- Confirm New Password (password type)

A red 'CHANGE PASSWORD' button is located at the bottom of the form.

## 11. Admin Dashboard:



## 12. Manage User:

The Manage Users interface allows administrators to manage the system's user base. It includes a sidebar with navigation options: Dashboard, ManageUser, Customers Details, Manage Vehicle, ManageFeedback, Reports, and Settings. A red LOGOUT button is located at the bottom of the sidebar.

**Manage Users Key Features:**

- ADD USER** button
- User List Table:**

| User ID | Customer Name | Role | Phone       | Actions |
|---------|---------------|------|-------------|---------|
| 2       | adm...        |      | 98951478621 |         |
| 5       | adm...        |      |             |         |
| 6       | roha...       |      |             |         |
| 10      | roha...       |      |             |         |
| 11      | ajay...       |      |             |         |
| 14      | har...        |      |             |         |
| 16      | faya...       |      |             |         |
| 17      | bha...        |      |             |         |
| 18      | raha...       |      |             |         |

**Add User Modal:**

- User Name \***: test@gmail.com
- Password \***: Min 8 chars, 1 uppercase, 1 lowercase, 1 number, 1 special char
- Role**: user
- Phone**: 6361521279
- Buttons**: CANCEL, ADD

## 13. Add Customer:

The screenshot shows the 'Customers Details' page with a sidebar menu on the left containing: Dashboard, ManageUser, Customers Details, Manage Vehicle, ManageFeedback, Reports, and Settings. A red 'LOGOUT' button is at the bottom of the sidebar. The main content area has a title 'Customers Details' and an 'ADD CUSTOMER' button. Below this is a table with columns: Customer ID, Customer Name, Address, Contact No, Email, and Vehicle No. An 'Add Customer' modal is open in the center, containing the following fields: Customer Name (dropdown with 'bhavya' selected), Contact No \* (text input with '6361521279'), Email \* (text input with 'bhavya@gmail.com'), Address \* (text input with 'Padvinagady Mangalore'), and Vehicle No \* (text input with 'Ka19mh8796'). The modal has 'CANCEL' and 'ADD' buttons at the bottom right. The background table shows several rows of customer data.

| Customer ID | Customer Name | Address | Contact No | Email               | Vehicle No  |
|-------------|---------------|---------|------------|---------------------|-------------|
| 1           | tes           |         |            | vignesh@gmail.com   | KA21Z100    |
| 2           | RC            |         |            | rXXXXXXXXXXXX@gm... | KA18EG7770  |
| 3           | ad            |         |            | roh@gmail.com       | MH12AB12355 |
| 4           | ad            |         |            | hi@gmail.com        | KA17JH8790  |
| 5           | aja           |         |            | ajay@gmail.com      | KA01MM1000  |
| 6           | fay           |         |            | fayaz@gmail.com     | KA21Z555    |
| 7           | bh            |         |            | bhavya@gmail.com    | KA21N3222   |
| 8           | rat           |         |            | pawan@gmail.com     | ka19hn9055  |
| 9           | rat           |         |            | rahul@gmail.com     | ka19hn9055  |

## 14. Manage Vehicle:

The screenshot shows the 'Manage Vehicle' page with a sidebar menu on the left containing: Dashboard, ManageUser, Customers Details, Manage Vehicle, ManageFeedback, Reports, and Settings. A red 'LOGOUT' button is at the bottom of the sidebar. The main content area has a title 'Manage Vehicle' and an 'ADD VEHICLE' button. Below this is a table with columns: Vehicle ID, Customer Name, Vehicle No, Model No, Insurance No, and Actions. An 'Add Vehicle' modal is open in the center, containing the following fields: Customer Name (dropdown with 'bhavya' selected), Vehicle Number \* (text input with 'Ka19mh8796'), Model Number (text input with 'vxi 2019'), and Insurance Number (text input with 'lxey76543'). The modal has 'CANCEL' and 'ADD' buttons at the bottom right. The background table shows several rows of vehicle data.

| Vehicle ID | Customer Name | Vehicle No | Model No | Insurance No | Actions                                                                                                                                                                     |
|------------|---------------|------------|----------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2          | tes           | KA21Z100   |          | 4780988      |   |
| 3          | RC            |            |          | 9            |   |
| 4          | ad            |            |          | 2            |   |

## 15. Manage Feedback:

The screenshot displays the 'Manage Feedback' interface. On the left is a sidebar with navigation links: Dashboard, ManageUser, Customers Details, Manage Vehicle, ManageFeedback, Reports, and Settings. A red 'LOGOUT' button is at the bottom of the sidebar. The main content area is titled 'Manage Feedback' and features an 'ADD FEEDBACK' button. Below this is a table with columns: ID, Name, Email, Feedback, Created On, and Actions. The table contains two rows of feedback data. A modal form titled 'Add Feedback' is open in the center, containing fields for Name (Bhavaya), Email (test@gmail.com), and a text area for 'Write Feedback' (Good Service). The modal has 'CANCEL' and 'ADD' buttons at the bottom. At the bottom right of the table, there is a pagination control showing 'Rows per page: 100' and '1-2 of 2'.

| ID | Name           | Email               | Feedback | Created On | Actions |
|----|----------------|---------------------|----------|------------|---------|
| 1  | Mohammad Ashik | ashikbk10@gmail.com | good     | 18/03/2025 |         |
| 2  | rahul          |                     |          | 18/03/2025 |         |

**Add Feedback**

Name \*  
Bhavaya

Email \*  
test@gmail.com

Write Feedback  
Good Service

CANCEL ADD

Rows per page: 100 1-2 of 2

## 16. Reports:

The screenshot displays the 'Employee Report' interface. On the left is a sidebar with navigation links: Dashboard, ManageUser, Customers Details, Manage Vehicle, ManageFeedback, Reports, and Settings. A red 'LOGOUT' button is at the bottom of the sidebar. The main content area is titled 'Employee Report' and features a 'Report Type' dropdown menu set to 'Employee Report'. To the right of the dropdown are three buttons: 'APPLY' (blue), 'RESET' (pink), and 'DOWNLOAD PDF' (green). Below these is a table with columns: Employee Name, User Type, Phone, and Email. The table contains one row of data. At the bottom right of the table, there is a pagination control showing 'Rows per page: 100' and '1-2 of 2'.

| Employee Name | User Type | Phone      | Email |
|---------------|-----------|------------|-------|
| employee      | employee  | 6361521279 |       |

Rows per page: 100 1-2 of 2

## 17. Settings:

Dashboard

ManageUser

Customers Details

Manage Vehicle

ManageFeedback

Reports

Settings

LOGOUT

Change Password

Current Password

New Password

Confirm New Password

CHANGE PASSWORD



---

## 11. CONCLUSION

I can confidently say that my internship at Accolade Tech Solutions has been a valuable learning experience that helped me improve both my technical skills and professional abilities. I'm especially grateful to the team for their guidance and support throughout this period, which helped me strengthen my frontend development skills using React JS and Material UI. Working on the Cafe Management System gave me the opportunity to apply what I've learned in a practical, real-world project.

During the internship, I built a solid understanding of modern web technologies like React JS, Material UI, and JavaScript. I gained hands-on experience in creating user interfaces, building functional components, and managing application states with React Hooks. Integrating MySQL as the backend database also taught me how to handle and organize data for users, orders, and wallet transactions effectively.

Along the way, I got familiar with important development concepts such as API integration, component-based design, and writing clean, modular code. These skills helped me in developing key features like order management, cart systems, wallet recharge functionalities, and user account controls for the system.

Overall, this internship gave me a great glimpse into the professional software development process, far beyond what I had learned in academics alone. I now feel more confident in my ability to build full-fledged web applications and better prepared to take on future challenges in the tech industry. I'm truly thankful for this opportunity and the growth it has brought me.

---

## 12. REFERENCES

- [1] <https://react.dev/>
- [2] <https://mui.com/material-ui/getting-started/>
- [3] <https://www.w3schools.com/>
- [4] <https://codepen.io/>
- [5] <https://nodejs.org/en>
- [6] <https://fakestoreapi.com/>
- [7] <https://dummyjson.com/>